Metropolitan University, Sylhet

Department of Computer Science & Engineering

Software Requirements Specification (SRS)

Version 1.0 — Final

Resume Builder AI

An AI-Powered Full-Stack Resume Creation Platform

Prepared by	Arup Ratan Dey
ID	231-115-013
Batch	CSE 58
Course	Software Engineering & Design Patterns Lab
Course Teacher	Md. Mushtaq Shahriyar Rafee
Institution	Metropolitan University, Sylhet
Date	20 April, 2026
Status	**Final**

This document contains the complete Software Requirements Specification for the Resume Builder AI web application.

Revision History

Version	Date	Description	Author
1.0	20 April 2026	Initial SRS document - complete draft covering all functional and non-functional requirements for Resume Builder AI v1.0.	Arup Ratan Dey

Contents

Revision History **1**

1 Introduction **4**

1.1 Purpose 4

1.2 Document Conventions 4

1.3 Intended Audience and Reading Suggestions 4

1.4 Project Scope 5

1.5 References 5

2 Overall Description **6**

2.1 Product Perspective 6

2.2 Product Features 7

2.3 User Classes and Characteristics 8

2.4 Operating Environment 8

2.5 Design and Implementation Constraints 8

2.6 User Documentation 8

2.7 Assumptions and Dependencies 9

3 System Features **10**

3.1 User Authentication 10

3.1.1 Description 10

3.1.2 Stimulus/Response Sequences 10

3.1.3 Functional Requirements 10

3.2 Resume Management Dashboard 11

3.2.1 Description 11

3.2.2 Stimulus/Response Sequences 11

3.2.3 Functional Requirements 12

3.3 AI-Powered Resume Upload 12

3.3.1 Description 12

3.3.2 Stimulus/Response Sequences 12

3.3.3 Functional Requirements 13

3.4 Multi-Section Resume Editor 13

3.4.1 Description 13

3.4.2 Functional Requirements 13

3.5 AI Content Enhancement 14

3.5.1 Description 14

3.5.2 Functional Requirements 14

3.6 Template Selection and Color Theming 15

3.6.1 Description 15

3.6.2 Functional Requirements 15

3.7 Profile Image Upload with Background Removal 15

3.7.1 Description 15

3.7.2 Functional Requirements 15

3.8 PDF Download 16

3.8.1 Description 16

3.8.2 Functional Requirements . . . 16
3.9 Public Resume Sharing . . . 16
3.9.1 Description . . . 16
3.9.2 Functional Requirements . . . 17
4 External Interface Requirements 18
4.1 User Interfaces . . . 18
4.2 Hardware Interfaces . . . 18
4.3 Software Interfaces . . . 19
4.4 Communications Interfaces . . . 19
5 Other Non-Functional Requirements 21
5.1 Performance Requirements . . . 21
5.2 Safety Requirements . . . 21
5.3 Security Requirements . . . 21
5.4 Software Quality Attributes . . . 22
6 Other Requirements 23
6.1 Database Requirements . . . 23
6.2 Internationalization Requirements . . . 23
6.3 Legal and Compliance Requirements . . . 23
6.4 Future Enhancements (Out of Scope for v1.0) . . . 23
Appendix A: System Architecture Overview 24

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document formally describes all functional and non-functional requirements for **Resume Builder AI** - a full-stack web application that empowers users to create, edit, manage, and export professional resumes with the assistance of AI-powered tools.

The purpose of this document is to act as an authoritative reference guiding the design, development, testing, and academic evaluation of the system. It covers **Version 1.0** of Resume Builder AI in its current implemented state. Core features documented include user authentication, resume creation and management, AI content enhancement, PDF export, and public resume sharing.

1.2 Document Conventions

The following conventions are used throughout this document:

- **Bold text** is used to highlight important terms, component names, and feature labels.
- `Monospace font` is used for technical identifiers, API endpoint paths, and code-related references.
- Requirements are labeled as **REQ-XX**, where XX is a two-digit sequential number.
- Priority levels are assigned as **High**, **Medium**, or **Low** and listed alongside each system feature.
- Higher-level requirements are assumed to pass their priority level down to child requirements unless stated otherwise.

1.3 Intended Audience and Reading Suggestions

Developers Should read the full document with special focus on Sections 3 (System Features) and 4 (External Interface Requirements).

Testers / QA Should focus on Sections 3 and 5 to derive test cases from functional and non-functional requirements.

Academic Evaluators Should read sequentially, beginning with Sections 1 and 2 for a system overview, then Sections 3 and 5 for details.

Project Managers Should review Section 2 for scope and Section 5 for quality and performance expectations.

> *Suggested reading order: Section 1 (Introduction) → Section 2 (Overall Description) → Section 3 (System Features) → Section 4 (Interfaces) → Section 5 (Non-Functional Requirements)*

1.4 Project Scope

Resume Builder AI is a cloud-connected, AI-integrated web application that allows authenticated users to:

- Create and manage multiple professional resumes with different templates and color themes.
- Upload existing PDF resumes and have the system automatically extract and populate resume data using AI.
- Enhance specific sections (professional summary, job descriptions) using AI-generated content.
- Download resumes as printable PDFs directly from the browser.
- Make resumes publicly accessible and share them via unique URLs.
- Upload and process profile images with optional AI background removal.

The system is built on a **MERN-adjacent** stack: React.js (frontend), Node.js with Express.js (backend), MongoDB with Mongoose (database), and integrates with Google Gemini AI via an OpenAI-compatible API and ImageKit for image hosting.

Out of scope for v1.0: payment/billing features, real-time collaborative editing, ATS scoring engines, and job application tracking.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- React.js Documentation: `https://react.dev`
- Express.js Documentation: `https://expressjs.com`
- MongoDB Documentation: `https://www.mongodb.com/docs`
- Google Gemini API (OpenAI-Compatible): `https://ai.google.dev/docs`
- ImageKit Documentation: `https://docs.imagekit.io`
- Redux Toolkit: `https://redux-toolkit.js.org`
- Tailwind CSS: `https://tailwindcss.com/docs`

2 Overall Description

2.1 Product Perspective

Resume Builder AI is a standalone, self-contained web application. It follows a **client-server architecture**:

- **Client (Frontend):** A Single Page Application (SPA) built with React.js, communicating with the backend via RESTful API calls over HTTP.
- **Server (Backend):** A RESTful API built with Node.js and Express.js that handles business logic, data persistence, authentication, and third-party service integration.
- **Database:** MongoDB Atlas (cloud-hosted) stores all user accounts and resume data via the Mongoose ODM.
- **External Services:** Google Gemini AI for content generation, and ImageKit for image upload, transformation, and CDN hosting.

Figure 1: System Architecture Diagram — Resume Builder AI

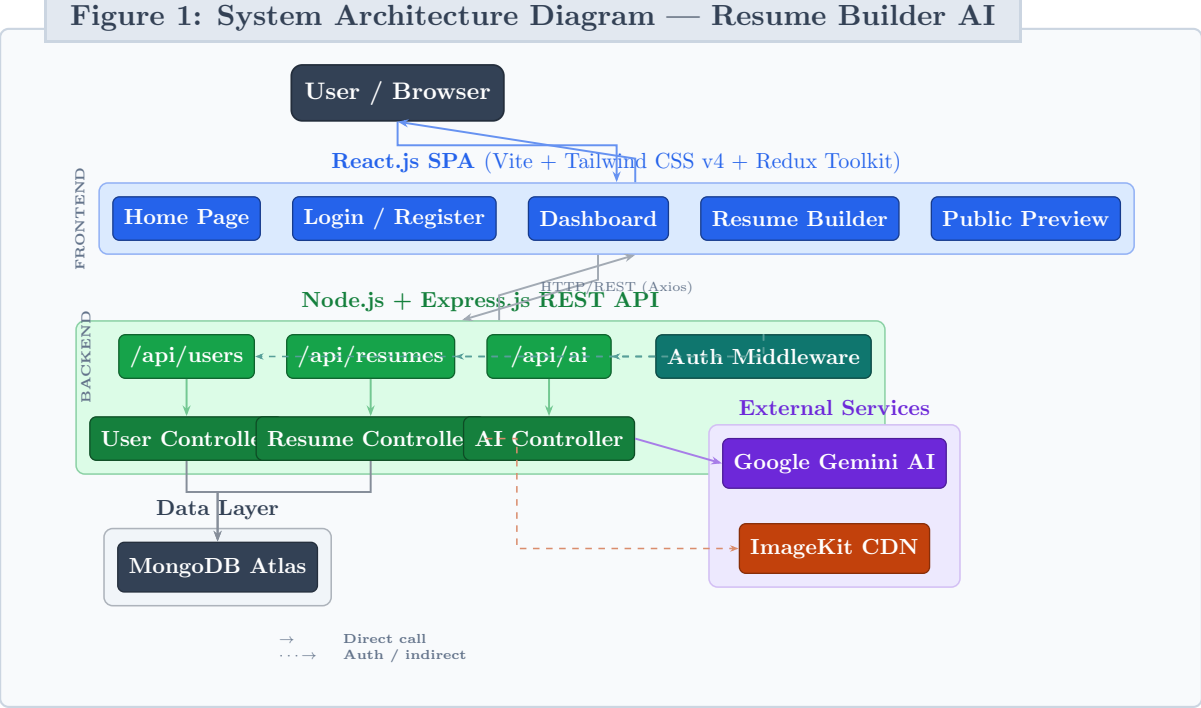
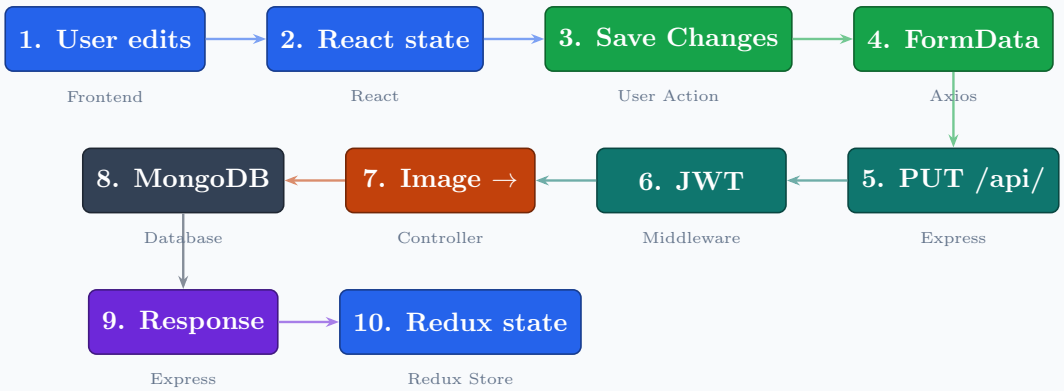


Figure 2: Data Flow Diagram — Resume Save Operation



2.2 Product Features

#	Feature	Description
F1	User Authentication	Secure registration and login using bcrypt-hashed passwords and JWT session tokens.
F2	Resume Management Dashboard	Create, view, rename, and delete multiple resumes per user.
F3	AI-Powered Resume Upload	Upload an existing PDF; AI extracts and populates structured resume data automatically.
F4	Multi-Section Resume Editor	Step-by-step form covering all resume sections with a live preview panel.
F5	AI Content Enhancement	Enhance professional summary and job descriptions using AI at the click of a button.
F6	Template Selection	Choose from four professional templates: Classic, Modern, Minimal, Minimal with Image.
F7	Color Theming	Select custom accent colors from a predefined palette for resume personalization.
F8	Profile Image Upload	Upload a profile photo with optional AI background removal via ImageKit.
F9	Live Resume Preview	Real-time preview reflecting all form changes instantly without saving.
F10	PDF Download	Export the resume as a print-formatted PDF using the browser's native print API.
F11	Public Resume Sharing	Toggle resume visibility to public and share via a unique, accessible URL.

2.3 User Classes and Characteristics

Registered User (Primary) The core user of the system. Has a verified account and access to all features. Assumed to have basic computer literacy; no technical background required.

Guest / Public Viewer A non-authenticated visitor who accesses a public resume via a shared URL. Read-only access; no account or login required.

Administrator (Future Scope) Not implemented in v1.0. Future versions may include an admin role for user and content management.

2.4 Operating Environment

- **Client:** Any modern web browser (Chrome 90+, Edge 90+, Firefox 88+, Safari 14+) on desktop or mobile. No installation required.
- **Server:** Node.js v20.x on Windows or Linux. Default port: 3000.
- **Database:** MongoDB Atlas (cloud-hosted, M0 free tier or above). Min version: MongoDB 6.0.
- **Network:** Active internet connection required for AI features, image hosting, and database operations.
- **External APIs:** Google Generative Language API (Gemini), ImageKit Node.js SDK.

2.5 Design and Implementation Constraints

- **AI Model Availability:** All AI features depend on Google Gemini API availability. Free-tier rate limits may affect responsiveness.
- **JWT Authentication:** Tokens expire after 7 days and are stored in `localStorage`. Not suitable for environments requiring stricter session management.
- **PDF Generation:** Uses `window.print()`, which relies on browser print behavior. Layout accuracy may vary across browsers.
- **Image Processing:** Background removal uses ImageKit's `e-bgremove` transformation, which requires a paid ImageKit plan.
- **No Server-Side Rendering:** The frontend is a pure client-side SPA, which may affect SEO for public resume pages.
- **CORS:** Backend applies permissive CORS in development. Production must restrict allowed origins.

2.6 User Documentation

- **In-Application Hints:** Descriptive placeholder text and contextual tips accompany each form field.
- **README (Developer Docs):** A `README.md` file documents project setup, environment variable configuration, and deployment steps.
- **This SRS Document:** Formal technical specification for the system.

2.7 Assumptions and Dependencies

- Users are assumed to have access to a reliable internet connection during usage.
- The system depends on the **Google Gemini API** being accessible with a valid API key and sufficient quota.
- The system depends on **MongoDB Atlas** being operational and the connection string in `.env` being correctly configured.
- The system depends on **ImageKit** for image upload and transformation; profile image features are non-functional without valid ImageKit credentials.
- Users' browsers are assumed to support `window.print()` and the `navigator.share()` Web Share API.
- The `react-pdftotext` library is assumed to correctly extract text from well-formatted PDF resumes. Scanned or image-based PDFs may yield poor results.

3 System Features

3.1 User Authentication

Priority: High

3.1.1 Description

The system provides secure user registration and login. Authentication is enforced on all protected routes. JWT tokens are issued upon successful login or registration and must be included in all subsequent protected API requests.

3.1.2 Stimulus/Response Sequences

Registration Flow:

1. User navigates to the login/register page and selects the Sign Up option.
2. User enters full name, email address, and password.
3. System validates required fields and checks for duplicate email.
4. System hashes the password with bcrypt (salt rounds: 10) and creates the user record.
5. System issues a JWT token (7-day expiry), stores it in `localStorage`, and redirects the user to the dashboard.

Login Flow:

1. User enters email and password on the login form and submits.
2. System verifies the email exists and the password matches via bcrypt comparison.
3. On success, a JWT token is issued and the user is redirected to the dashboard.
4. On failure, a generic error is shown to prevent email enumeration attacks.

Session Persistence:

1. On application load, the system reads the token from `localStorage`.
2. Token is used to call `GET /api/users/data` to retrieve user data.
3. If valid, the dashboard is displayed. If invalid or missing, the login page is shown.

3.1.3 Functional Requirements

REQ-01 The system shall allow users to register with a unique email address, a full name, and a password.

REQ-02 The system shall reject registration if the email is already associated with an existing account and return an appropriate error message.

REQ-03 The system shall store passwords as bcrypt hashes with salt factor 10. Plain-text passwords shall never be stored or logged.

REQ-04 The system shall issue a signed JWT token upon successful login or registration, valid for 7 days.

REQ-05 All protected API routes shall require a valid JWT token in the Authorization header. Requests without a valid token shall receive a 401 Unauthorized response.

REQ-06 The system shall support user logout by clearing the token from `localStorage` and resetting the Redux authentication state.

REQ-07 The system shall automatically restore the user session from `localStorage` on page load if a valid token exists.

3.2 Resume Management Dashboard

Priority: High

3.2.1 Description

The dashboard is the central hub where authenticated users view all their resumes, create new ones, upload existing PDFs, edit resume titles, and delete resumes. Each resume appears as a colored card showing its title and last-updated date.

3.2.2 Stimulus/Response Sequences

Create Resume:

1. User clicks the Create Resume button on the dashboard.
2. A modal appears prompting the user to enter a resume title.
3. System creates a new resume in the database with default empty values.
4. User is redirected to the Resume Builder editor for the new resume.

Delete Resume:

1. User hovers over a card; delete and edit icons become visible.
2. User clicks the delete icon; a browser confirmation dialog appears.
3. On confirmation, a DELETE request is sent; the resume is removed from the database.

Edit Resume Title:

1. User clicks the edit icon on a card; a modal with the current title appears.
2. User modifies the title and submits.
3. The title is updated in the database and reflected immediately on the card.

3.2.3 Functional Requirements

REQ-08 The system shall display all resumes belonging to the currently authenticated user on the dashboard page.

REQ-09 Each resume card shall display the resume title and the date it was last updated.

REQ-10 The system shall allow users to create a new resume by providing a title. The resume shall be initialized with default empty values for all sections.

REQ-11 The system shall allow users to permanently delete a resume after explicit confirmation via a browser confirmation dialog.

REQ-12 The system shall allow users to rename a resume by editing its title via a modal form.

REQ-13 Each resume card shall be assigned a distinct accent color from a predefined palette for visual differentiation.

3.3 AI-Powered Resume Upload

Priority: High

3.3.1 Description

Users can upload an existing PDF resume. The system extracts text from the PDF on the client side using `react-pdftotext` and sends it to the backend, where Google Gemini AI parses the text into structured resume data.

3.3.2 Stimulus/Response Sequences

1. User clicks Upload Existing Resume on the dashboard.
2. A modal appears for the user to enter a title and select a PDF file.
3. The frontend extracts plain text from the PDF using `react-pdftotext`.
4. The extracted text and title are sent to `POST /api/ai/upload-resume`.
5. The backend sends the text to Google Gemini AI with a structured JSON extraction prompt.
6. Gemini returns parsed resume data as a structured JSON object.
7. The backend creates a new resume document in MongoDB using the parsed data.
8. The user is redirected to the Resume Builder editor for the newly created resume.

3.3.3 Functional Requirements

REQ-14 The system shall accept PDF file uploads from authenticated users for AI-based resume parsing.

REQ-15 The system shall use Google Gemini AI to extract structured resume data from the uploaded PDF's text content.

REQ-16 The extracted data shall be mapped to the defined resume schema including: personal information, work experience, education, and projects.

REQ-17 The system shall display a loading indicator during the upload and AI processing period.

REQ-18 If the AI fails to parse the PDF, the system shall display an appropriate error message to the user.

3.4 Multi-Section Resume Editor

Priority: High

3.4.1 Description

The Resume Builder is a step-by-step multi-section form allowing users to fill in and edit all parts of their resume. Sections are navigated via Previous and Next buttons. A progress bar indicates the current section, and all changes reflect instantly in the live preview panel.

3.4.2 Functional Requirements

REQ-19 The editor shall contain the following sections in order: Personal Information, Professional Summary, Experience, Education, Projects, and Skills.

REQ-20 Users shall be able to navigate between sections using clearly labeled Previous and Next buttons.

REQ-21 A visual progress bar shall indicate the user's current position within the multi-step form.

REQ-22 The Personal Information section shall include fields for: full name, profession, email, phone, location, LinkedIn URL, website URL, and profile image upload.

REQ-23 The Experience section shall allow adding, editing, and removing multiple work experience entries. Each entry shall include: company name, job title, start date, end date, description, and a currently working here toggle.

REQ-24 The Education section shall allow adding multiple entries with: institution, degree, field of study, graduation date, and optional GPA.

REQ-25 The Projects section shall allow adding multiple entries with: project name, type, and description.

REQ-26 The Skills section shall allow users to add individual skill tags by typing and pressing Enter or clicking Add, and to remove individual skill tags.

REQ-27 All changes in the editor shall be reflected in the live resume preview in real time without requiring an explicit save.

REQ-28 Users shall save changes explicitly using the Save Changes button, which persists data to the backend via `PUT /api/resumes/update`.

3.5 AI Content Enhancement

Priority: High

3.5.1 Description

Users can enhance their Professional Summary and individual job descriptions using AI. The system sends the existing content to Google Gemini, which returns an improved, ATS-friendly version that replaces the original text in the editor.

3.5.2 Functional Requirements

REQ-29 The system shall provide an AI Enhance button in the Professional Summary section.

REQ-30 Clicking AI Enhance shall send the current summary text to `POST /api/ai/enhance-pro-sum` and replace the textarea content with the AI-generated result.

REQ-31 The system shall provide an Enhance with AI button for each individual work experience job description field.

REQ-32 The AI job description enhancement button shall be disabled unless both the company name and job title fields of that entry are filled in.

REQ-33 During AI generation, the enhance button shall display a loading spinner and be disabled to prevent duplicate requests.

REQ-34 AI-generated content shall be concise (1–2 sentences), ATS-friendly, and use professional action verbs.

3.6 Template Selection and Color Theming

Priority: Medium

3.6.1 Description

Users can select from four distinct resume templates and customize the accent color from a palette of available options. Both selections are saved to the database and reflected immediately in the live preview.

3.6.2 Functional Requirements

REQ-35 The system shall offer four resume templates: Classic, Modern, Minimal, and Minimal with Image.

REQ-36 The user's template selection shall be persisted to the database and loaded on subsequent visits.

REQ-37 The system shall offer a color picker with at least 10 accent color options.

REQ-38 The selected accent color shall be applied consistently to headings, section borders, and skill tags across all templates.

REQ-39 Template and color changes shall be reflected in the live preview immediately without requiring an explicit save.

3.7 Profile Image Upload with Background Removal

Priority: Medium

3.7.1 Description

Users can upload a profile picture for their resume. The image is processed by ImageKit with face-focused cropping and optional AI background removal. The resulting hosted image URL is stored and displayed in templates that support profile photos.

3.7.2 Functional Requirements

REQ-40 The system shall allow users to upload a profile image (JPEG or PNG) within the Personal Information section.

REQ-41 The uploaded image shall be processed by ImageKit with face-focused cropping at 300×300 pixels.

REQ-42 The user shall have the option to toggle AI background removal before saving. This toggle shall only be visible when a new image file has been selected.

REQ-43 The processed image URL returned from ImageKit shall be stored in the resume's `personal_info.image` field.

REQ-44 A live preview of the selected image shall be shown in the editor before saving.

3.8 PDF Download

Priority: High

3.8.1 Description

Users can download their resume as a PDF using the browser's built-in print functionality. Print-specific CSS ensures only the resume content is rendered, formatted for letter-size paper.

3.8.2 Functional Requirements

REQ-45 The system shall provide a Download button in the Resume Builder that triggers `window.print()` when clicked.

REQ-46 The print stylesheet shall hide all application UI elements and render only the resume preview content.

REQ-47 The print layout shall be formatted for US Letter paper size (8.5 × 11 inches) with zero margins.

REQ-48 Accent colors and background fills shall be preserved during printing using `-webkit-print-color-adjust: exact`.

3.9 Public Resume Sharing

Priority: Medium

3.9.1 Description

Users can toggle their resume to Public visibility, making it accessible via a unique URL without authentication. A Share button appears for public resumes, allowing users to share the link via the Web Share API.

3.9.2 Functional Requirements

REQ-49 Each resume shall have a public/private visibility toggle. Resumes shall be private by default upon creation.

REQ-50 Public resumes shall be accessible at the route `/view/:resumeId` without requiring authentication.

REQ-51 The `GET /api/resumes/public/:resumeId` endpoint shall not require a JWT token and shall return data only if `public` is `true`.

REQ-52 When a resume is set to public, a Share button shall appear. Clicking it shall invoke `navigator.share()` with the public resume URL.

REQ-53 The public resume viewer shall display the full resume using the same template and accent color settings saved by the owner.

4 External Interface Requirements

4.1 User Interfaces

- **Login / Register Page:** Centered card form supporting both modes via a link toggle. Fields: Name (register only), Email, Password.
- **Dashboard Page:** Responsive resume card grid with create and upload buttons. Cards reveal delete and edit icons on hover.
- **Resume Builder Page:** Two-column layout — left panel: multi-step form with section navigation, template selector, color picker, and save button; right panel: live preview with visibility toggle, share, and download controls.
- **Public Resume View:** A minimal centered page displaying the resume with no navigation or authentication elements.
- **Home Page (Landing):** Marketing page with Hero, Features, Testimonials, Call-to-Action, and Footer sections. Responsive for desktop and mobile.
- **Notifications:** Toast notifications via `react-hot-toast` provide feedback for all major success and error events.

4.2 Hardware Interfaces

The application is entirely web-based and requires no proprietary hardware. Standard hardware interactions include:

- **File System:** Accessed via `<input type="file">` for PDF uploads and profile image selection.
- **Printer:** The browser's native print system is used for PDF generation via `window.print()`.

4.3 Software Interfaces

Service	Purpose	Integration Method
MongoDB Atlas	Persistent storage of users and resumes	Mongoose ODM via `MONGODB_URI` env variable
Google Gemini AI	AI text generation and resume extraction	`openai` npm package at Gemini's OpenAI-compatible endpoint
ImageKit SDK	Image upload, crop, background removal	`@imagekit/nodejs` SDK with transformation parameters
Multer	Parsing `multipart/form-data` file uploads	Express middleware with disk storage
jsonwebtoken	Signing and verifying auth tokens	npm package using `JWT_SECRET` env variable
bcrypt	Hashing and comparing passwords	npm package with salt factor 10
react-pdftotext	Extracting plain text from PDFs on the client	Client-side React library; no server dependency

4.4 Communications Interfaces

- **Protocol:** HTTP/HTTPS. Frontend communicates via RESTful HTTP using Axios.
- **Base URL:** Configured via the `VITE_BASE_URL` environment variable (e.g., `http://localhost:3000/`
- **Auth Header:** JWT tokens are passed in the `Authorization` header.
- **Data Format:** JSON for structured data; `multipart/form-data` for file uploads.
- **CORS:** Permissive in development. Must be restricted to the frontend domain in production.

Key API Endpoints:

Method	Endpoint	Auth?	Description
POST	`/api/users/register`	No	Register a new user account
POST	`/api/users/login`	No	Authenticate and receive a JWT token
GET	`/api/users/data`	Yes	Get current authenticated user's data
GET	`/api/users/resumes`	Yes	Get all resumes for the current user
POST	`/api/resumes/create`	Yes	Create a new empty resume
PUT	`/api/resumes/update`	Yes	Update resume data (supports image upload)
DELETE	`/api/resumes/delete/:id`	Yes	Delete a resume by ID
GET	`/api/resumes/get/:id`	Yes	Get a specific resume (authenticated)
GET	`/api/resumes/public/:id`	No	Get a public resume (no auth required)
POST	`/api/ai/enhance-pro-sum`	Yes	AI-enhance professional summary
POST	`/api/ai/enhance-job-desc`	Yes	AI-enhance job description
POST	`/api/ai/upload-resume`	Yes	Parse PDF resume text using AI

5 Other Non-Functional Requirements

5.1 Performance Requirements

REQ-54 Standard CRUD operations (create, read, update, delete resume) shall complete within 2 seconds under normal network conditions.

REQ-55 AI enhancement requests (Gemini API calls) shall complete within 10 seconds. A loading indicator shall be shown throughout this period.

REQ-56 The dashboard shall load and render all user resumes within 3 seconds of page load.

REQ-57 Live resume preview updates shall reflect editor changes within 100 milliseconds, as React state updates are synchronous and immediate.

REQ-58 PDF upload and AI parsing shall complete within 30 seconds. A disabled submit button shall prevent duplicate submissions during processing.

5.2 Safety Requirements

REQ-59 The system shall confirm all destructive operations (resume deletion) with a browser confirmation dialog before executing.

REQ-60 AI-generated content shall be presented as a suggestion and shall not overwrite existing user data without an explicit user action.

REQ-61 Image uploads shall be validated on the client side to accept only JPEG and PNG file formats.

5.3 Security Requirements

REQ-62 Passwords shall be hashed using bcrypt with salt factor 10 before storage. Plain-text passwords shall never appear in logs or database records.

REQ-63 All protected API endpoints shall verify the JWT token on every request. Expired or malformed tokens shall result in a 401 Unauthorized response.

REQ-64 All resume CRUD operations shall be scoped to the authenticated user's `userId`. A user shall not be able to access or modify another user's resumes.

REQ-65 API keys and secrets (JWT secret, MongoDB URI, Gemini key, ImageKit private key) shall be stored in environment variables and must never be committed to version control.

REQ-66 Public resume endpoints shall only expose data for resumes explicitly set to `public: true`. Private resumes shall not be accessible without authentication.

REQ-67 In production, the CORS policy shall restrict allowed origins to the registered frontend domain only.

5.4 Software Quality Attributes

Attribute	Description
Usability	The application shall be usable without prior training. Navigation is intuitive via labeled Previous/Next buttons and a progress bar. Error messages are human-readable and toast notifications confirm all major actions.
Maintainability	Clear separation of concerns: routes, controllers, models, and middleware in separate directories. The frontend separates pages, components, and configuration files.
Reliability	All API controllers use try-catch blocks. The system returns appropriate HTTP status codes (400, 401, 404, 500) with descriptive messages. The frontend shows toast error messages for all failed requests.
Portability	Deployable on any Node.js 20+ server. Frontend is a static Vite build deployable to Vercel, Netlify, etc. MongoDB Atlas requires no local database setup.
Scalability	Stateless JWT authentication allows horizontal backend scaling. MongoDB Atlas supports scaling via replica sets and sharding.
Testability	Controllers are pure async functions suitable for unit testing with Jest. API endpoints can be tested with Postman or Supertest.

6 Other Requirements

6.1 Database Requirements

The system uses two primary MongoDB collections:

- **Users Collection:** Stores user credentials and profile data. Fields: `name`, `email` (unique, indexed), `password` (bcrypt hash), `createdAt`, `updatedAt`.
- **Resumes Collection:** Stores all resume data per user. Key fields: `userId` (ref to Users), `title`, `public` (boolean), `template`, `accent_color`, `professional_summary`, `skills` (array), `personal_info` (sub-document), `experience` (array), `project` (array), `education` (array), `createdAt`, `updatedAt`. The schema uses `minimize: false` to preserve empty objects during serialization.

6.2 Internationalization Requirements

Version 1.0 is developed entirely in English. No multi-language support is included. Future versions may incorporate i18n support using `react-i18next`.

6.3 Legal and Compliance Requirements

- User data (names, emails, resume content) is stored in MongoDB Atlas. Handling must comply with applicable data protection regulations (e.g., GDPR) in production.
- AI-generated content is produced by Google Gemini. Users are responsible for reviewing and validating AI content before using it in a professional context.

6.4 Future Enhancements (Out of Scope for v1.0)

- Multi-language support for resume content and the application interface.
- ATS (Applicant Tracking System) score analysis and improvement suggestions.
- Real-time collaborative editing for teams.
- Email-based password reset functionality.
- Additional resume templates (timeline, two-column, infographic).
- Server-side PDF rendering using Puppeteer for consistent cross-browser output.
- Admin dashboard for user account and content management.

Appendix A: System Architecture Overview

Layer	Technology	Details
Frontend	React.js 19 + Vite	SPA, Tailwind CSS v4, Redux Toolkit, React Router v7, Axios, react-hot-toast
Backend	Node.js v20 + Express.js 5	RESTful API, ES Modules, JWT auth, bcrypt, Multer, CORS
Database	MongoDB Atlas + Mongoose	Cloud NoSQL, two collections: Users and Resumes
AI Service	Google Gemini 2.0 Flash	Via OpenAI-compatible endpoint. Resume extraction, summary and job description enhancement
Image Service	ImageKit (`@imagekit/nodejs`)	Profile image upload, face-crop (300×300), optional AI background removal
PDF Generation	Browser `window.print()`	Native print API with `@media print` CSS rules

Requirements Traceability Summary

Requirement IDs	System Feature	Section
REQ-01 to REQ-07	User Authentication	3.1
REQ-08 to REQ-13	Resume Management Dashboard	3.2
REQ-14 to REQ-18	AI-Powered Resume Upload	3.3
REQ-19 to REQ-28	Multi-Section Resume Editor	3.4
REQ-29 to REQ-34	AI Content Enhancement	3.5
REQ-35 to REQ-39	Template Selection & Color Theming	3.6
REQ-40 to REQ-44	Profile Image Upload	3.7
REQ-45 to REQ-48	PDF Download	3.8
REQ-49 to REQ-53	Public Resume Sharing	3.9
REQ-54 to REQ-58	Performance Requirements	5.1
REQ-59 to REQ-61	Safety Requirements	5.2
REQ-62 to REQ-67	Security Requirements	5.3